

For instructions on how to download the sample, see [Sample applications for Windows development](#).

Common control behaviors

Touch controls and mouse/keyboard controls have a very similar core implementation. In a UWP app, a pointer is simply a point on the screen. You can move it by sliding the mouse or sliding your finger on the touch screen. As a result, you can register for a single set of events, and not worry about whether the player is using a mouse or a touch screen to move and press the pointer.

When the **MoveLookController** class in the sample game is initialized, it registers for four pointer-specific events and one mouse-specific event:

 Expand table

Event	Description
CoreApplication::PointerPressed	The left or right mouse button was pressed (and held), or the touch surface was touched.
CoreApplication::PointerMoved	The mouse moved, or a drag action was made on the touch surface.
CoreApplication::PointerReleased	The left mouse button was released, or the object contacting the touch surface was lifted.
CoreApplication::PointerExited	The pointer moved out of the main window.
Windows::Devices::Input::MouseMoved	The mouse moved a certain distance. Be aware that we are only interested in mouse movement delta values, and not the current X-Y position.

These event handlers are set to start listening for user input as soon as the **MoveLookController** is initialized in the application window.

C++/WinRT

```
void MoveLookController::InitWindow(_In_ CoreWindow const& window)
{
    ResetState();

    window.PointerPressed({ this, &MoveLookController::OnPointerPressed });
    window.PointerMoved({ this, &MoveLookController::OnPointerMoved });
    window.PointerReleased({ this, &MoveLookController::OnPointerReleased
```

```
});

window.PointerExited({ this, &MoveLookController::OnPointerExited });

...

// There is a separate handler for mouse-only relative mouse movement
events.
MouseDevice::GetForCurrentView().MouseMoved({ this,
&MoveLookController::OnMouseMoved });

...
}
```

Complete code for [InitWindow](#) can be seen on GitHub.

To determine when the game should be listening for certain input, the **MoveLookController** class has three controller-specific states, regardless of the controller type:

[Expand table](#)

State	Description
None	This is the initialized state for the controller. All input is ignored since the game is not anticipating any controller input.
WaitForInput	The controller is waiting for the player to acknowledge a message from the game by either using a left mouse click, a touch event, or the menu button on a gamepad.
Active	The controller is in active game play mode.

WaitForInput state and pausing the game

The game enters the **WaitForInput** state when the game has been paused. This happens when the player moves the pointer outside the main window of the game, or presses the pause button (the P key or the gamepad **Start** button). The **MoveLookController** registers the press, and informs the game loop when it calls the [IsPauseRequested](#) method. At that point if **IsPauseRequested** returns **true**, the game loop then calls **WaitForPress** on the **MoveLookController** to move the controller into the **WaitForInput** state.

Once in the **WaitForInput** state, the game stops processing almost all gameplay input events until it returns to the **Active** state. The exception is the pause button, with a press of this causing the game to go back to the active state. Other than the pause button, in

order for the game to go back to the **Active** state the player needs to select a menu item.

The Active state

During the **Active** state, the **MoveLookController** instance is processing events from all enabled input devices and interpreting the player's intentions. As a result, it updates the velocity and look direction of the player's view and shares the updated data with the game after **Update** is called from the game loop.

All pointer input is tracked in the **Active** state, with different pointer IDs corresponding to different pointer actions. When a **PointerPressed** event is received, the **MoveLookController** obtains the pointer ID value created by the window. The pointer ID represents a specific type of input. For example, on a multi-touch device, there may be several different active inputs at the same time. The IDs are used to keep track of which input the player is using. If one event is in the move rectangle of the touch screen, a pointer ID is assigned to track any pointer events in the move rectangle. Other pointer events in the fire rectangle are tracked separately, with a separate pointer ID.

ⓘ Note

Input from the mouse and right thumbstick of a gamepad also have IDs that are handled separately.

After the pointer events have been mapped to a specific game action, it's time to update the data the **MoveLookController** object shares with the main game loop.

When called, the **Update** [↗](#) method in the sample game processes the input and updates the velocity and look direction variables (**m_velocity** and **m_lookdirection**), which the game loop then retrieves by calling the public **Velocity** [↗](#) and **LookDirection** [↗](#) methods.

ⓘ Note

More details about the **Update** method can be seen later on this page.

The game loop can test to see if the player is firing by calling the **IsFiring** method on the **MoveLookController** instance. The **MoveLookController** checks to see if the player has pressed the fire button on one of the three input types.

```

bool MoveLookController::IsFiring()
{
    if (m_state == MoveLookControllerState::Active)
    {
        if (m_autoFire)
        {
            return (m_fireInUse || (m_mouseInUse && m_mouseLeftInUse) ||
PollingFireInUse());
        }
        else
        {
            if (m_firePressed)
            {
                m_firePressed = false;
                return true;
            }
        }
    }
    return false;
}

```

Now let's look at the implementation of each of the three control types in a little more detail.

Adding relative mouse controls

If mouse movement is detected, we want to use that movement to determine the new pitch and yaw of the camera. We do that by implementing relative mouse controls, where we handle the relative distance the mouse has moved—the delta between the start of the movement and the stop—as opposed to recording the absolute x-y pixel coordinates of the motion.

To do that, we obtain the changes in the X (the horizontal motion) and the Y (the vertical motion) coordinates by examining the [MouseDelta::X](#) and [MouseDelta::Y](#) fields on the [Windows::Device::Input::MouseEventArgs::MouseDelta](#) argument object returned by the [MouseMoved](#) event.

C++/WinRT

```

void MoveLookController::OnMouseMoved(
    _In_ MouseDevice const& /* mouseDevice */,
    _In_ MouseEventArgs const& args
)
{
    // Handle Mouse Input via dedicated relative movement handler.

    switch (m_state)

```

```

{
case MoveLookControllerState::Active:
    XMFLOAT2 mouseDelta;
    mouseDelta.x = static_cast<float>(args.MouseDelta().X);
    mouseDelta.y = static_cast<float>(args.MouseDelta().Y);

    XMFLOAT2 rotationDelta;
    // Scale for control sensitivity.
    rotationDelta.x = mouseDelta.x * MoveLookConstants::RotationGain;
    rotationDelta.y = mouseDelta.y * MoveLookConstants::RotationGain;

    // Update our orientation based on the command.
    m_pitch -= rotationDelta.y;
    m_yaw += rotationDelta.x;

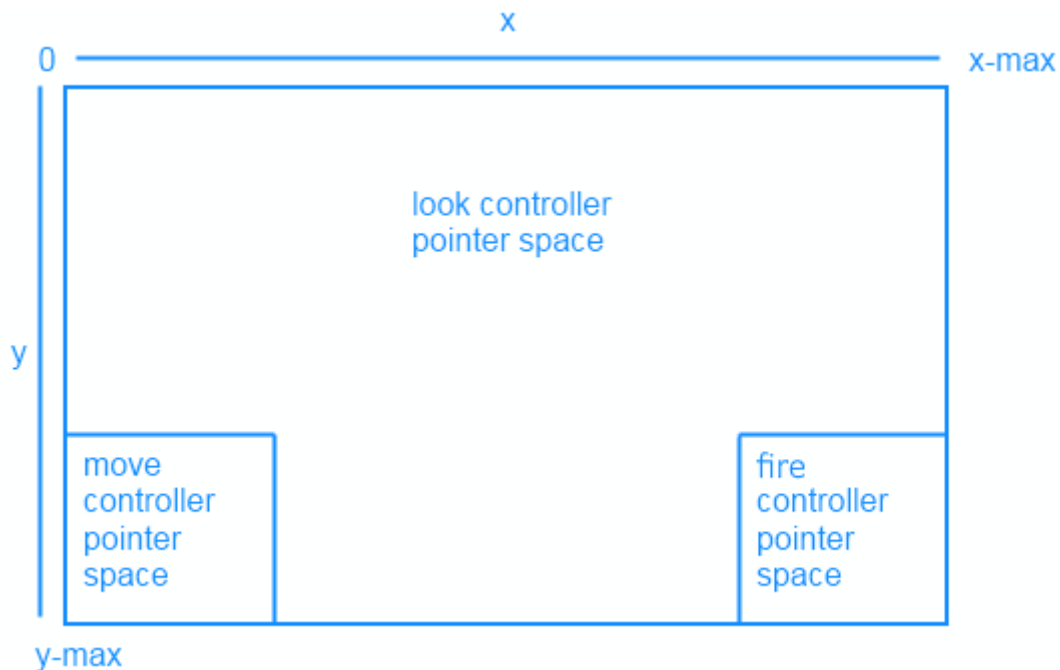
    // Limit pitch to straight up or straight down.
    float limit = XM_PI / 2.0f - 0.01f;
    m_pitch = __max(-limit, m_pitch);
    m_pitch = __min(+limit, m_pitch);

    // Keep longitude in sane range by wrapping.
    if (m_yaw > XM_PI)
    {
        m_yaw -= XM_PI * 2.0f;
    }
    else if (m_yaw < -XM_PI)
    {
        m_yaw += XM_PI * 2.0f;
    }
    break;
}
}

```

Adding touch support

Touch controls are great for supporting users with tablets. This game gathers touch input by zoning off certain areas of the screen with each aligning to specific in-game actions. This game's touch input uses three zones.



The following commands summarize our touch control behavior. User input | Action :---
 --- | :----- Move rectangle | Touch input is converted into a virtual joystick where the vertical motion will be translated into forward/backward position motion and horizontal motion will be translated into left/right position motion. Fire rectangle | Fire a sphere.
 Touch outside of move and fire rectangle | Change the rotation (the pitch and yaw) of the camera view.

The **MoveLookController** checks the pointer ID to determine where the event occurred, and takes one of the following actions:

- If the **PointerMoved** event occurred in the move or fire rectangle, update the pointer position for the controller.
- If the **PointerMoved** event occurred somewhere in the rest of the screen (defined as the look controls), calculate the change in pitch and yaw of the look direction vector.

Once we've implemented our touch controls, the rectangles we drew earlier using Direct2D will indicate to players where the move, fire, and look zones are.



Now let's take a look at how we implement each control.

Move and fire controller

The move controller rectangle in the lower left quadrant of the screen is used as a directional pad. Sliding your thumb left and right within this space moves the player left and right, while up and down moves the camera forward and backward. After setting this up, tapping the fire controller in the lower right quadrant of the screen fires a sphere.

The [SetMoveRect](#) and [SetFireRect](#) methods create our input rectangles, taking two, 2D vectors to specify each rectangles' upper left and lower right corner positions on the screen.

The parameters are then assigned to **m_fireUpperLeft** and **m_fireLowerRight** that will help us determine if the user is touching within on of the rectangles.

C++/WinRT

```
m_fireUpperLeft = upperLeft;  
m_fireLowerRight = lowerRight;
```

If the screen is resized, these rectangles are redrawn to the appropriate size.

Now that we've zoned off our controls, it's time to determine when a user is actually using them. To do this, we set up some event handlers in the **MoveLookController::InitWindow** method for when the user presses, moves, or releases their pointer.

C++/WinRT

```
window.PointerPressed({ this, &MoveLookController::OnPointerPressed });  
  
window.PointerMoved({ this, &MoveLookController::OnPointerMoved });  
  
window.PointerReleased({ this, &MoveLookController::OnPointerReleased });
```

We'll first determine what happens when the user first presses within the move or fire rectangles using the [OnPointerPressed](#) method. Here we check where they're touching a control and if a pointer is already in that controller. If this is the first finger to touch the specific control, we do the following.

- Store the location of the touchdown in **m_moveFirstDown** or **m_fireFirstDown** as a 2D vector.
- Assign the pointer ID to **m_movePointerID** or **m_firePointerID**.
- Set the proper **InUse** flag (**m_moveInUse** or **m_fireInUse**) to `true` since we now have an active pointer for that control.

C++/WinRT

```
PointerPoint point = args.CurrentPoint();  
uint32_t pointerID = point.PointerId();  
Point pointerPosition = point.Position();  
PointerPointProperties pointProperties = point.Properties();  
auto pointerDevice = point.PointerDevice();  
auto pointerDeviceType = pointerDevice.PointerDeviceType();  
  
XMFLOAT2 position = XMFLOAT2(pointerPosition.X, pointerPosition.Y);  
  
...  
case MoveLookControllerState::Active:  
    switch (pointerDeviceType)  
    {  
    case winrt::Windows::Devices::Input::PointerDeviceType::Touch:  
        // Check to see if this pointer is in the move control.  
        if (position.x > m_moveUpperLeft.x &&  
            position.x < m_moveLowerRight.x &&  
            position.y > m_moveUpperLeft.y &&  
            position.y < m_moveLowerRight.y)  
        {  
            // If no pointer is in this control yet.  
            if (!m_moveInUse)  
            {
```



```

        // Process a DPad touch down event.
        // Save the location of the initial contact
        m_moveFirstDown = position;
        // Store the pointer using this control
        m_movePointerID = pointerID;
        // Set InUse flag to signal there is an active move pointer
        m_moveInUse = true;
    }
}
// Check to see if this pointer is in the fire control.
else if (position.x > m_fireUpperLeft.x &&
    position.x < m_fireLowerRight.x &&
    position.y > m_fireUpperLeft.y &&
    position.y < m_fireLowerRight.y)
{
    if (!m_fireInUse)
    {
        // Save the location of the initial contact
        m_fireLastPoint = position;
        // Store the pointer using this control
        m_firePointerID = pointerID;
        // Set InUse flag to signal there is an active fire pointer
        m_fireInUse = true;
        ...
    }
}
...

```

Now that we've determined whether the user is touching a move or fire control, we see if the player is making any movements with their pressed finger. Using the [MoveLookController::OnPointerMoved](#) method, we check what pointer has moved and then store its new position as a 2D vector.

C++/WinRT

```

PointerPoint point = args.CurrentPoint();
uint32_t pointerID = point.PointerId();
Point pointerPosition = point.Position();
PointerPointProperties pointProperties = point.Properties();
auto pointerDevice = point.PointerDevice();

// convert to allow math
XMFLOAT2 position = XMFLOAT2(pointerPosition.X, pointerPosition.Y);

switch (m_state)
{
case MoveLookControllerState::Active:
    // Decide which control this pointer is operating.

    // Move control
    if (pointerID == m_movePointerID)
    {

```

```

        // Save the current position.
        m_movePointerPosition = position;
    }
    // Look control
    else if (pointerID == m_lookPointerID)
    {
        ...
    }
    // Fire control
    else if (pointerID == m_firePointerID)
    {
        m_fireLastPoint = position;
    }
    ...

```

Once the user has made their gestures within the controls, they'll release the pointer. Using the [MoveLookController::OnPointerReleased](#) method, we determine which pointer has been released and do a series of resets.

If the move control has been released, we do the following.

- Set the velocity of the player to `0` in all directions to prevent them from moving in the game.
- Switch `m_moveInUse` to `false` since the user is no longer touching the move controller.
- Set the move pointer ID to `0` since there's no longer a pointer in the move controller.

C++/WinRT

```

if (pointerID == m_movePointerID)
{
    // Stop on release.
    m_velocity = XMFLOAT(0, 0, 0);
    m_moveInUse = false;
    m_movePointerID = 0;
}

```

For the fire control, if it has been released all we do is switch the `m_fireInUse` flag to `false` and the fire pointer ID to `0` since there's no longer a pointer in the fire control.

C++/WinRT

```

else if (pointerID == m_firePointerID)
{
    m_fireInUse = false;
}

```

```
m_firePointerID = 0;
}
```

Look controller

We treat touch device pointer events for the unused regions of the screen as the look controller. Sliding your finger around this zone changes the pitch and yaw (rotation) of the player camera.

If the **MoveLookController::OnPointerPressed** event is raised on a touch device in this region and the game state is set to **Active**, it's assigned a pointer ID.

C++/WinRT

```
// If no pointer is in this control yet.
if (!m_lookInUse)
{
    // Save point for later move.
    m_lookLastPoint = position;
    // Store the pointer using this control.
    m_lookPointerID = pointerID;
    // These are for smoothing.
    m_lookLastDelta.x = m_lookLastDelta.y = 0;
    m_lookInUse = true;
}
```

Here the **MoveLookController** assigns the pointer ID for the pointer that fired the event to a specific variable that corresponds to the look region. In the case of a touch occurring in the look region, the **m_lookPointerID** variable is set to the pointer ID that fired the event. A boolean variable, **m_lookInUse**, is also set to indicate that the control has not yet been released.

Now, let's look at how the sample game handles the **PointerMoved** touch screen event.

Within the **MoveLookController::OnPointerMoved** method, we check to see what kind of pointer ID has been assigned to the event. If it's **m_lookPointerID**, we calculate the change in position of the pointer. We then use this delta to calculate how much the rotation should change. Finally we're at a point where we can update the **m_pitch** and **m_yaw** to be used in the game to change the player rotation.

C++/WinRT

```
// This is the look pointer.
else if (pointerID == m_lookPointerID)
{
    // Look control.
```

```

XMFLOAT2 pointerDelta;
// How far did the pointer move?
pointerDelta.x = position.x - m_lookLastPoint.x;
pointerDelta.y = position.y - m_lookLastPoint.y;

XMFLOAT2 rotationDelta;
// Scale for control sensitivity.
rotationDelta.x = pointerDelta.x * MoveLookConstants::RotationGain;
rotationDelta.y = pointerDelta.y * MoveLookConstants::RotationGain;
// Save for next time through.
m_lookLastPoint = position;

// Update our orientation based on the command.
m_pitch -= rotationDelta.y;
m_yaw += rotationDelta.x;

// Limit pitch to straight up or straight down.
float limit = XM_PI / 2.0f - 0.01f;
m_pitch = __max(-limit, m_pitch);
m_pitch = __min(+limit, m_pitch);
...
}

```

The last piece we'll look at is how the sample game handles the [PointerReleased](#) touch screen event. Once the user has finished the touch gesture and removed their finger from the screen, [MoveLookController::OnPointerReleased](#) [↗](#) is initiated. If the ID of the pointer that fired the [PointerReleased](#) event is the ID of the previously recorded move pointer, the **MoveLookController** sets the velocity to `0` because the player has stopped touching the look area.

C++/WinRT

```

else if (pointerID == m_lookPointerID)
{
    m_lookInUse = false;
    m_lookPointerID = 0;
}

```

Adding mouse and keyboard support

This game has the following control layout for keyboard and mouse.

[↗](#) Expand table

User input	Action
W	Move player forward

User input	Action
A	Move player left
S	Move player backward
D	Move player right
X	Move view up
Space bar	Move view down
P	Pause the game
Mouse movement	Change the rotation (the pitch and yaw) of the camera view
Left mouse button	Fire a sphere

To use the keyboard, the sample game registers two new events, [CoreWindow::KeyUp](#) and [CoreWindow::KeyDown](#), within the [MoveLookController::InitWindow](#) method. These events handle the press and release of a key.

C++/WinRT

```
window.KeyDown({ this, &MoveLookController::OnKeyDown });
window.KeyUp({ this, &MoveLookController::OnKeyUp });
```

The mouse is treated a little differently from the touch controls even though it uses a pointer. To align with our control layout, the **MoveLookController** rotates the camera whenever the mouse is moved, and fires when the left mouse button is pressed.

This is handled in the [OnPointerPressed](#) method of the **MoveLookController**.

In this method we check to see what type of pointer device is being used with the [Windows::Devices::Input::PointerDeviceType](#) enum. If the game is **Active** and the **PointerDeviceType** isn't **Touch**, we assume it's mouse input.

C++/WinRT

```
case MoveLookControllerState::Active:
    switch (pointerDeviceType)
    {
        case winrt::Windows::Devices::Input::PointerDeviceType::Touch:
            // Behavior for touch controls
            ...

        default:
            // Behavior for mouse controls
```

```

bool rightButton = pointProperties.IsRightButtonPressed();
bool leftButton = pointProperties.IsLeftButtonPressed();

if (!m_autoFire && (!m_mouseLeftInUse && leftButton))
{
    m_firePressed = true;
}

if (!m_mouseInUse)
{
    m_mouseInUse = true;
    m_mouseLastPoint = position;
    m_mousePointerID = pointerID;
    m_mouseLeftInUse = leftButton;
    m_mouseRightInUse = rightButton;
    // These are for smoothing.
    m_lookLastDelta.x = m_lookLastDelta.y = 0;
}
break;
}
break;

```

When the player stops pressing one of the mouse buttons, the `CoreWindow::PointerReleased` mouse event is raised, calling the `MoveLookController::OnPointerReleased` [method](#), and the input is complete. At this point, spheres will stop firing if the left mouse button was being pressed and is now released. Because look is always enabled, the game continues to use the same mouse pointer to track the ongoing look events.

C++/WinRT

```

case MoveLookControllerState::Active:
    // Touch points
    if (pointerID == m_movePointerID)
    {
        // Stop movement
        ...
    }
    else if (pointerID == m_lookPointerID)
    {
        // Stop look rotation
        ...
    }
    // Fire button has been released
    else if (pointerID == m_firePointerID)
    {
        // Stop firing
        ...
    }
    // Mouse point
    else if (pointerID == m_mousePointerID)
    {

```

```

    bool rightButton = pointProperties.IsRightButtonPressed();
    bool leftButton = pointProperties.IsLeftButtonPressed();

    // Mouse no longer in use so stop firing
    m_mouseInUse = false;

    // Don't clear the mouse pointer ID so that Move events still result
    in Look changes.
    // m_mousePointerID = 0;
    m_mouseLeftInUse = leftButton;
    m_mouseRightInUse = rightButton;
}
break;

```

Now let's look at the last control type we'll be supporting: gamepads. Gamepads are handled separately from the touch and mouse controls since they doesn't use the pointer object. Because of this, a few new event handlers and methods will need to be added.

Adding gamepad support

For this game, gamepad support is added by calls to the [Windows.Gaming.Input](#) APIs. This set of APIs provides access to game controller inputs like racing wheels and flight sticks.

The following will be our gamepad controls.

[Expand table](#)

User input	Action
Left analog stick	Move player
Right analog stick	Change the rotation (the pitch and yaw) of the camera view
Right trigger	Fire a sphere
Start/Menu button	Pause or resume the game

In the [InitWindow](#) method, we add two new events to determine if a gamepad has been [added](#) or [removed](#). These events update the `m_gamepadsChanged` property. This is used in the `UpdatePollingDevices` method to check if the list of known gamepads has changed.

```
// Detect gamepad connection and disconnection events.
Gamepad::GamepadAdded({ this, &MoveLookController::OnGamepadAdded });

Gamepad::GamepadRemoved({ this, &MoveLookController::OnGamepadRemoved });
```

ⓘ Note

UWP apps cannot receive input from a game controller while the app is not in focus.

The UpdatePollingDevices method

The [UpdatePollingDevices](#) method of the **MoveLookController** instance immediately checks to see if a gamepad is connected. If one is, we'll start reading its state with [Gamepad.GetCurrentReading](#). This returns the [GamepadReading](#) struct, allowing us to check what buttons have been clicked or thumbsticks moved.

If the state of the game is **WaitForInput**, we only listen for the Start/Menu button of the controller so that the game can be resumed.

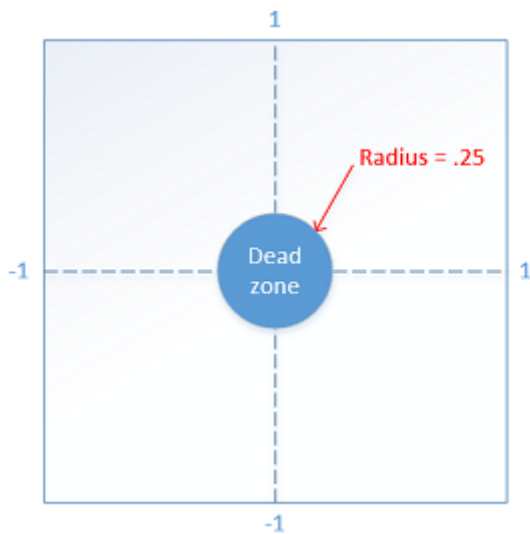
If it's **Active**, we check the user's input and determine what in-game action needs to happen. For instance, if the user moved the left analog stick in a specific direction, this lets the game know we need to move the player in the direction the stick is being moved. The movement of the stick in a specific direction must register as larger than the radius of the **dead zone**; otherwise, nothing will happen. This dead zone radius is necessary to prevent "drifting," which is when the controller picks up small movements from the player's thumb as it rests on the stick. Without dead zones, the controls can appear too sensitive to the user.

Thumbstick input is between -1 and 1 for both the x and y axis. The following constant specifies the radius of the thumbstick dead zone.

C++/WinRT

```
#define THUMBSTICK_DEADZONE 0.25f
```

Using this variable, we'll then begin processing actionable thumbstick input. Movement would occur with a value from [-1, -.26] or [.26, 1] on either axis.



This piece of the **UpdatePollingDevices** method handles the left and right thumbsticks. Each stick's X and Y values are checked to see if they are outside of the dead zone. If one or both are, we'll update the corresponding component. For example, if the left thumbstick is being moved left along the X axis, we'll add -1 to the x component of the **m_moveCommand** vector. This vector is what will be used to aggregate all movements across all devices and will later be used to calculate where the player should move.

C++/WinRT

```
// Use the left thumbstick to control the eye point position
// (position of the player).

// Check if left thumbstick is outside of dead zone on x axis
if (reading.LeftThumbstickX > THUMBSTICK_DEADZONE ||
    reading.LeftThumbstickX < -THUMBSTICK_DEADZONE)
{
    // Get value of left thumbstick's position on x axis
    float x = static_cast<float>(reading.LeftThumbstickX);
    // Set the x of the move vector to 1 if the stick is being moved right.
    // Set to -1 if moved left.
    m_moveCommand.x -= (x > 0) ? 1 : -1;
}

// Check if left thumbstick is outside of dead zone on y axis
if (reading.LeftThumbstickY > THUMBSTICK_DEADZONE ||
    reading.LeftThumbstickY < -THUMBSTICK_DEADZONE)
{
    // Get value of left thumbstick's position on y axis
    float y = static_cast<float>(reading.LeftThumbstickY);
    // Set the y of the move vector to 1 if the stick is being moved
    forward.
    // Set to -1 if moved backwards.
    m_moveCommand.y += (y > 0) ? 1 : -1;
}
```

Similar to how the left stick controls movement, the right stick controls the rotation of the camera.

The right thumb stick behavior aligns with the behavior of mouse movement in our mouse and keyboard control setup. If the stick is outside of the dead zone, we calculate the difference between the current pointer position and where the user is now trying to look. This change in pointer position (**pointerDelta**) is then used to update the pitch and yaw of the camera rotation that later get applied in our **Update** method. The **pointerDelta** vector may look familiar because it's also used in the [MoveLookController::OnPointerMoved](#) method to keep track of change in pointer position for our mouse and touch inputs.

C++/WinRT

```
// Use the right thumbstick to control the look at position

XMFLOAT2 pointerDelta;

// Check if right thumbstick is outside of deadzone on x axis
if (reading.RightThumbstickX > THUMBSTICK_DEADZONE ||
    reading.RightThumbstickX < -THUMBSTICK_DEADZONE)
{
    float x = static_cast<float>(reading.RightThumbstickX);
    // Register the change in the pointer along the x axis
    pointerDelta.x = x * x * x;
}
// No actionable thumbstick movement. Register no change in pointer.
else
{
    pointerDelta.x = 0.0f;
}
// Check if right thumbstick is outside of deadzone on y axis
if (reading.RightThumbstickY > THUMBSTICK_DEADZONE ||
    reading.RightThumbstickY < -THUMBSTICK_DEADZONE)
{
    float y = static_cast<float>(reading.RightThumbstickY);
    // Register the change in the pointer along the y axis
    pointerDelta.y = y * y * y;
}
else
{
    pointerDelta.y = 0.0f;
}

XMFLOAT2 rotationDelta;
// Scale for control sensitivity.
rotationDelta.x = pointerDelta.x * 0.08f;
rotationDelta.y = pointerDelta.y * 0.08f;

// Update our orientation based on the command.
m_pitch += rotationDelta.y;
```

```
m_yaw += rotationDelta.x;

// Limit pitch to straight up or straight down.
m_pitch = __max(-XM_PI / 2.0f, m_pitch);
m_pitch = __min(+XM_PI / 2.0f, m_pitch);
```

The game's controls wouldn't be complete without the ability to fire spheres!

This **UpdatePollingDevices** method also checks if the right trigger is being pressed. If it is, our **m_firePressed** property is flipped to true, signaling to the game that spheres should start firing.

C++/WinRT

```
if (reading.RightTrigger > TRIGGER_DEADZONE)
{
    if (!m_autoFire && !m_gamepadTriggerInUse)
    {
        m_firePressed = true;
    }

    m_gamepadTriggerInUse = true;
}
else
{
    m_gamepadTriggerInUse = false;
}
```

The Update method

To wrap things up, let's dig deeper into the [Update](#) method. This method merges any movements or rotations that the player made with any supported input to generate a velocity vector and update our pitch and yaw values for our game loop to access.

The **Update** method kicks things off by calling [UpdatePollingDevices](#) to update the state of the controller. This method also gathers any input from a gamepad and adds its movements to the **m_moveCommand** vector.

In our **Update** method we then perform the following input checks.

- If the player is using the move controller rectangle, we'll then determine the change in pointer position and use that to calculate if the user has moved the pointer out of the controller's dead zone. If outside of the dead zone, the **m_moveCommand** vector property is then updated with the virtual joystick value.
- If any of the movement keyboard inputs are pressed, a value of `1.0f` or `-1.0f` are added in the corresponding component of the **m_moveCommand** vector—`1.0f`

for forward, and `-1.0f` for backward.

Once all movement input has been taken into account, we then run the `m_moveCommand` vector through some calculations to generate a new vector that represents the direction of the player with regards to the game world. We then take our movements in relation to the world and apply them to the player as velocity in that direction. Finally we reset the `m_moveCommand` vector to `(0.0f, 0.0f, 0.0f)` so that everything is ready for the next game frame.

C++/WinRT

```
void MoveLookController::Update()
{
    // Get any gamepad input and update state
    UpdatePollingDevices();

    if (m_moveInUse)
    {
        // Move control.
        XMFLOAT2 pointerDelta;

        pointerDelta.x = m_movePointerPosition.x - m_moveFirstDown.x;
        pointerDelta.y = m_movePointerPosition.y - m_moveFirstDown.y;

        // Figure out the command from the virtual joystick.
        XMFLOAT3 commandDirection = XMFLOAT3(0.0f, 0.0f, 0.0f);
        // Leave 32 pixel-wide dead spot for being still.
        if (fabsf(pointerDelta.x) > 16.0f)
            m_moveCommand.x -= pointerDelta.x / fabsf(pointerDelta.x);

        if (fabsf(pointerDelta.y) > 16.0f)
            m_moveCommand.y -= pointerDelta.y / fabsf(pointerDelta.y);
    }

    // Poll our state bits set by the keyboard input events.
    if (m_forward)
    {
        m_moveCommand.y += 1.0f;
    }
    if (m_back)
    {
        m_moveCommand.y -= 1.0f;
    }
    if (m_left)
    {
        m_moveCommand.x += 1.0f;
    }
    if (m_right)
    {
        m_moveCommand.x -= 1.0f;
    }
    if (m_up)
```

```

{
    m_moveCommand.z += 1.0f;
}
if (m_down)
{
    m_moveCommand.z -= 1.0f;
}

// Make sure that 45deg cases are not faster.
if (fabsf(m_moveCommand.x) > 0.1f ||
    fabsf(m_moveCommand.y) > 0.1f ||
    fabsf(m_moveCommand.z) > 0.1f)
{
    XMStoreFloat3(&m_moveCommand,
XMVector3Normalize(XMLoadFloat3(&m_moveCommand)));
}

// Rotate command to align with our direction (world coordinates).
XMFLOAT3 wCommand;
wCommand.x = m_moveCommand.x * cosf(m_yaw) - m_moveCommand.y *
sinf(m_yaw);
wCommand.y = m_moveCommand.x * sinf(m_yaw) + m_moveCommand.y *
cosf(m_yaw);
wCommand.z = m_moveCommand.z;

// Scale for sensitivity adjustment.
// Our velocity is based on the command. Y is up.
m_velocity.x = -wCommand.x * MoveLookConstants::MovementGain;
m_velocity.z = wCommand.y * MoveLookConstants::MovementGain;
m_velocity.y = wCommand.z * MoveLookConstants::MovementGain;

// Clear movement input accumulator for use during next frame.
m_moveCommand = XMFLOAT3(0.0f, 0.0f, 0.0f);
}

```

Next steps

Now that we have added our controls, there's another feature we need to add to create an immersive game: sound! Music and sound effects are important to any game, so let's discuss [adding sound](#) next.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Add sound

Article • 10/20/2022

ⓘ Note

This topic is part of the [Create a simple Universal Windows Platform \(UWP\) game with DirectX](#) tutorial series. The topic at that link sets the context for the series.

In this topic, we create a simple sound engine using [XAudio2](#) APIs. If you are new to [XAudio2](#), we have included a short intro under [Audio concepts](#).

ⓘ Note

If you haven't downloaded the latest game code for this sample, go to [Direct3D sample game](#) [↗](#). This sample is part of a large collection of UWP feature samples. For instructions on how to download the sample, see [Sample applications for Windows development](#).

Objective

Add sounds into the sample game using [XAudio2](#).

Define the audio engine

In the sample game, the audio objects and behaviors are defined in three files:

- [Audio.h/.cpp](#): Defines the **Audio** object, which contains the **XAudio2** resources for sound playback. It also defines the method for suspending and resuming audio playback if the game is paused or deactivated.
- [MediaReader.h/.cpp](#): Defines the methods for reading audio .wav files from local storage.
- [SoundEffect.h/.cpp](#): Defines an object for in-game sound playback.

Overview

There are three main parts in getting set up for audio playback into your game.

1. [Create and initialize the audio resources](#)

2. [Load audio file](#)
3. [Associate sound to object](#)

They are all defined in the [Simple3DGame::Initialize](#) method. So let's first examine this method and then dive into more details in each of the sections.

After setting up, we learn how to trigger the sound effects to play. For more info, go to [Play the sound](#).

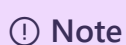
Simple3DGame::Initialize method

In [Simple3DGame::Initialize](#), where `m_controller` and `m_renderer` are also initialized, we set up the audio engine and get it ready to play sounds.

- Create `m_audioController`, which is an instance of the [Audio](#) class.
- Create the audio resources needed using the [Audio::CreateDeviceIndependentResources](#) method. Here, two `XAudio2` objects — a music engine object and a sound engine object, and a mastering voice for each of them were created. The music engine object can be used to play background music for your game. The sound engine can be used to play sound effects in your game. For more info, see [Create and initialize the audio resources](#).
- Create `mediaReader`, which is an instance of [MediaReader](#) class. [MediaReader](#), which is a helper class for the [SoundEffect](#) class, reads small audio files synchronously from file location and returns sound data as a byte array.
- Use [MediaReader::LoadMedia](#) to load sound files from its location and create a `targetHitSound` variable to hold the loaded .wav sound data. For more info, see [Load audio file](#).

Sound effects are associated with the game object. So when a collision occurs with that game object, it triggers the sound effect to be played. In this sample game, we have sound effects for the ammo (what we use to shoot targets with) and for the target.

- In the `GameObject` class, there's a `HitSound` property that is used to associate the sound effect to the object.
- Create a new instance of the [SoundEffect](#) class and initialize it. During initialization, a source voice for the sound effect is created.
- This class plays a sound using a mastering voice provided from the [Audio](#) class. Sound data is read from file location using the [MediaReader](#) class. For more info, see [Associate sound to object](#).



The actual trigger to play the sound is determined by the movement and collision of these game objects. Hence, the call to actually play these sounds are defined in the `Simple3DGame::UpdateDynamics` method. For more info, go to [Play the sound](#).

C++/WinRT

```
void Simple3DGame::Initialize(
    _In_ std::shared_ptr<MoveLookController> const& controller,
    _In_ std::shared_ptr<GameRenderer> const& renderer
)
{
    // The following member is defined in the header file:
    // Audio m_audioController;

    ...

    // Create the audio resources needed.
    // Two XAudio2 objects are created - one for music engine,
    // the other for sound engine. A mastering voice is also
    // created for each of the objects.
    m_audioController.CreateDeviceIndependentResources();

    m_ammo.resize(GameConstants::MaxAmmo);

    ...

    // Create a media reader which is used to read audio files from its file
    location.
    MediaReader mediaReader;
    auto targetHitSoundX = mediaReader.LoadMedia(L"Assets\\hit.wav");

    // Instantiate the targets for use in the game.
    // Each target has a different initial position, size, and orientation.
    // But share a common set of material properties.
    for (int a = 1; a < GameConstants::MaxTargets; a++)
    {
        ...
        // Create a new sound effect object and associate it
        // with the game object's (target) HitSound property.
        target->HitSound(std::make_shared<SoundEffect>());

        // Initialize the sound effect object with
        // the sound effect engine, format of the audio wave, and audio data
        // During initialization, source voice of this sound effect is also
        created.
        target->HitSound()->Initialize(
            m_audioController.SoundEffectEngine(),
            mediaReader.GetOutputWaveFormatEx(),
            targetHitSoundX
        );

        ...
    }
}
```



```

}

// Instantiate a set of spheres to be used as ammunition for the game
// and set the material properties of the spheres.
auto ammoHitSound = mediaReader.LoadMedia(L"Assets\\bounce.wav");

for (int a = 0; a < GameConstants::MaxAmmo; a++)
{
    m_ammo[a] = std::make_shared<Sphere>();
    m_ammo[a]->Radius(GameConstants::AmmoRadius);
    m_ammo[a]->HitSound(std::make_shared<SoundEffect>());
    m_ammo[a]->HitSound()->Initialize(
        m_audioController.SoundEffectEngine(),
        mediaReader.GetOutputWaveFormatEx(),
        ammoHitSound
    );
    m_ammo[a]->Active(false);
    m_renderObjects.push_back(m_ammo[a]);
}
...
}

```

Create and initialize the audio resources

- Use [XAudio2Create](#), an XAudio2 API, to create two new XAudio2 objects which define the music and sound effect engines. This method returns a pointer to the object's [IXAudio2](#) interface that manages all audio engine states, the audio processing thread, the voice graph, and more.
- After the engines have been instantiated, use [IXAudio2::CreateMasteringVoice](#) to create a mastering voice for each of the sound engine objects.

For more info, go to [How to: Initialize XAudio2](#).

Audio::CreateDeviceIndependentResources method

C++/WinRT

```

void Audio::CreateDeviceIndependentResources()
{
    UINT32 flags = 0;

    winrt::check_hresult(
        XAudio2Create(m_musicEngine.put(), flags)
    );

    HRESULT hr = m_musicEngine-
>CreateMasteringVoice(&m_musicMasteringVoice);
    if (FAILED(hr))

```

```

{
    // Unable to create an audio device
    m_audioAvailable = false;
    return;
}

winrt::check_hresult(
    XAudio2Create(m_soundEffectEngine.put(), flags)
);

winrt::check_hresult(
    m_soundEffectEngine->CreateMasteringVoice(&m_soundEffectMasteringVoice)
);

m_audioAvailable = true;
}

```

Load audio file

In the sample game, the code for reading audio format files is defined in [MediaReader.h/cpp](#). To read an encoded .wav audio file, call [MediaReader::LoadMedia](#), passing in the filename of the .wav as the input parameter.

MediaReader::LoadMedia method

This method uses the [Media Foundation](#) APIs to read in the .wav audio file as a Pulse Code Modulation (PCM) buffer.

Set up the Source Reader

1. Use [MFCreateSourceReaderFromURL](#) to create a media source reader ([IMFSourceReader](#)).
2. Use [MFCreateMediaType](#) to create a media type ([IMFMediaType](#)) object (*mediaType*). It represents a description of a media format.
3. Specify that the *mediaType*'s decoded output is PCM audio, which is an audio type that [XAudio2](#) can use.
4. Sets the decoded output media type for the source reader by calling [IMFSourceReader::SetCurrentMediaType](#).

For more info on why we use the Source Reader, go to [Source Reader](#).

Describe the data format of the audio stream

1. Use [IMFSourceReader::GetCurrentMediaType](#) to get the current media type for the stream.
2. Use [IMFMediaType::MFCreateWaveFormatExFromMFMediaType](#) to convert the current audio media type to a [WAVEFORMATEX](#) buffer, using the results of the earlier operation as input. This structure specifies the data format of the wave audio stream that is used after audio is loaded.

The **WAVEFORMATEX** format can be used to describe the PCM buffer. As compared to the [WAVEFORMATEXTENSIBLE](#) structure, it can only be used to describe a subset of audio wave formats. For more info about the differences between **WAVEFORMATEX** and **WAVEFORMATEXTENSIBLE**, see [Extensible Wave-Format Descriptors](#).

Read the audio stream

1. Get the duration, in seconds, of the audio stream by calling [IMFSourceReader::GetPresentationAttribute](#) and then converts the duration to bytes.
2. Read the audio file in as a stream by calling [IMFSourceReader::ReadSample](#). **ReadSample** reads the next sample from the media source.
3. Use [IMFSample::ConvertToContiguousBuffer](#) to copy contents of the audio sample buffer (*sample*) into an array (*mediaBuffer*).

C++/WinRT

```
std::vector<byte> MediaReader::LoadMedia(_In_ winrt::hstring const&
filename)
{
    winrt::check_hresult(
        MFStartup(MF_VERSION)
    );

    // Creates a media source reader.
    winrt::com_ptr<IMFSourceReader> reader;
    winrt::check_hresult(
        MFCreateSourceReaderFromURL(
            (m_installedLocationPath + filename).c_str(),
            nullptr,
            reader.put()
        )
    );

    // Set the decoded output format as PCM.
    // XAudio2 on Windows can process PCM and ADPCM-encoded buffers.
    // When using MediaFoundation, this sample always decodes into PCM.
    winrt::com_ptr<IMFMediaType> mediaType;
    winrt::check_hresult(
        MFCreateMediaType(mediaType.put())
    );
}
```

```

    );

    // Define the major category of the media as audio. For more info about
    major media types,
    // go to:
https://msdn.microsoft.com/library/windows/desktop/aa367377.aspx
    winrt::check_hresult(
        mediaType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Audio)
    );

    // Define the sub-type of the media as uncompressed PCM audio. For more
    info about audio sub-types,
    // go to:
https://msdn.microsoft.com/library/windows/desktop/aa372553.aspx
    winrt::check_hresult(
        mediaType->SetGUID(MF_MT_SUBTYPE, MFAudioFormat_PCM)
    );

    // Sets the media type for a stream. This media type defines that format
    that the Source Reader
    // produces as output. It can differ from the native format provided by
    the media source.
    // For more info, go to
https://msdn.microsoft.com/library/windows/desktop/dd374667.aspx
    winrt::check_hresult(
        reader->SetCurrentMediaType(static\_cast<uint32\_t>
(MF_SOURCE_READER_FIRST_AUDIO_STREAM), 0, mediaType.get())
    );

    // Get the current media type for the stream.
    // For more info, go to:
    // https://msdn.microsoft.com/library/windows/desktop/dd374660.aspx
    winrt::com_ptr<IMFMediaType> outputMediaType;
    winrt::check_hresult(
        reader->GetCurrentMediaType(static\_cast<uint32\_t>
(MF_SOURCE_READER_FIRST_AUDIO_STREAM), outputMediaType.put())
    );

    // Converts the current media type into the WaveFormatEx buffer
    structure.
    UINT32 size = 0;
    WAVEFORMATEX* waveFormat;
    winrt::check_hresult(
        MFCreateWaveFormatExFromMFMediaType(outputMediaType.get(),
&waveFormat, &size)
    );

    // Copies the waveFormat's block of memory to the starting address of
    the m_waveFormat variable in MediaReader.
    // Then free the waveFormat memory block.
    // For more info, go to
https://msdn.microsoft.com/library/windows/desktop/aa366535.aspx and
    // https://msdn.microsoft.com/library/windows/desktop/ms680722.aspx
    CopyMemory(&m_waveFormat, waveFormat, sizeof\(m\_waveFormat\));
    CoTaskMemFree(waveFormat);

```

```

    PROPVARIANT propVariant;
    winrt::check_hresult(
        reader->GetPresentationAttribute(static_cast<uint32_t>
(MF_SOURCE_READER_MEDIASOURCE), MF_PD_DURATION, &propVariant)
    );

    // 'duration' is in 100ns units; convert to seconds, and round up
    // to the nearest whole byte.
    LONGLONG duration = propVariant.uhVal.QuadPart;
    unsigned int maxStreamLengthInBytes =
        static_cast<unsigned int>(
            ((duration * static_cast<ULONGLONG>
(m_waveFormat.nAvgBytesPerSec)) + 10000000) /
            10000000
        );

    std::vector<byte> fileData(maxStreamLengthInBytes);

    winrt::com_ptr<IMFSample> sample;
    winrt::com_ptr<IMFMediaBuffer> mediaBuffer;
    DWORD flags = 0;

    int positionInData = 0;
    bool done = false;
    while (!done)
    {
        // Read audio data.
        ...
    }

    return fileData;
}

```

Associate sound to object

Associating sounds to the object takes place when the game initializes, in the [Simple3DGame::Initialize](#) method.

Recap:

- In the **GameObject** class, there's a **HitSound** property that is used to associate the sound effect to the object.
- Create a new instance of the [SoundEffect](#) class object and associate it with the game object. This class plays a sound using **XAudio2** APIs. It uses a mastering voice provided by the [Audio](#) class. The sound data can be read from file location using the [MediaReader](#) class.

[SoundEffect::Initialize](#) is used to initialize the **SoundEffect** instance with the following input parameters: pointer to sound engine object (IXAudio2 objects created in the [Audio::CreateDeviceIndependentResources](#) method), pointer to format of the .wav file using **MediaReader::GetOutputWaveFormatEx**, and the sound data loaded using [MediaReader::LoadMedia](#) method. During initialization, the source voice for the sound effect is also created.

SoundEffect::Initialize method

C++/WinRT

```
void SoundEffect::Initialize(
    _In_ IXAudio2* masteringEngine,
    _In_ WAVEFORMATEX* sourceFormat,
    _In_ std::vector<byte> const& soundData)
{
    m_soundData = soundData;

    if (masteringEngine == nullptr)
    {
        // Audio is not available so just return.
        m_audioAvailable = false;
        return;
    }

    // Create a source voice for this sound effect.
    winrt::check_hresult(
        masteringEngine->CreateSourceVoice(
            &m_sourceVoice,
            sourceFormat
        )
    );
    m_audioAvailable = true;
}
```

Play the sound

Triggers to play sound effects are defined in [Simple3DGame::UpdateDynamics](#) method because this is where movement of the objects are updated and collision between objects is determined.

Since interaction of between objects differs greatly, depending on the game, we are not going to discuss the dynamics of the game objects here. If you're interested to understand its implementation, go to [Simple3DGame::UpdateDynamics](#) method.

In principle, when a collision occurs, it triggers the sound effect to play by calling **SoundEffect::PlaySound**. This method stops any sound effects that's currently playing and queues the in-memory buffer with the desired sound data. It uses source voice to set the volume, submit sound data, and start the playback.

SoundEffect::PlaySound method

- Uses the source voice object **m_sourceVoice** to start the playback of the sound data buffer **m_soundData**
- Creates an **XAUDIO2_BUFFER**, to which it provides a reference to the sound data buffer, and then submits it with a call to **IXAudio2SourceVoice::SubmitSourceBuffer**.
- With the sound data queued up, **SoundEffect::PlaySound** starts play back by calling **IXAudio2SourceVoice::Start**.

C++/WinRT

```
void SoundEffect::PlaySound(_In_ float volume)
{
    XAUDIO2_BUFFER buffer = { 0 };

    if (!m_audioAvailable)
    {
        // Audio is not available so just return.
        return;
    }

    // Interrupt sound effect if it is currently playing.
    winrt::check_hresult(
        m_sourceVoice->Stop()
    );
    winrt::check_hresult(
        m_sourceVoice->FlushSourceBuffers()
    );

    // Queue the memory buffer for playback and start the voice.
    buffer.AudioBytes = (UINT32)m_soundData.size();
    buffer.pAudioData = m_soundData.data();
    buffer.Flags = XAUDIO2_END_OF_STREAM;

    winrt::check_hresult(
        m_sourceVoice->SetVolume(volume)
    );
    winrt::check_hresult(
        m_sourceVoice->SubmitSourceBuffer(&buffer)
    );
    winrt::check_hresult(
        m_sourceVoice->Start()
    );
}
```

Simple3DGame::UpdateDynamics method

The **Simple3DGame::UpdateDynamics** method takes care the interaction and collision between game objects. When objects collide (or intersect), it triggers the associated sound effect to play.

C++/WinRT

```
void Simple3DGame::UpdateDynamics()
{
    ...
    // Check for collisions between ammo.
#pragma region inter-ammo collision detection
    if (m_ammoCount > 1)
    {
        ...
        // Check collision between instances One and Two.
        ...
        if (distanceSquared < (GameConstants::AmmoSize *
GameConstants::AmmoSize))
        {
            // The two ammo are intersecting.
            ...
            // Start playing the sounds for the impact between the two balls.
            m_ammo[one]->PlaySound(impact, m_player->Position());
            m_ammo[two]->PlaySound(impact, m_player->Position());
        }
    }
#pragma endregion

#pragma region Ammo-Object intersections
    // Check for intersections between the ammo and the other objects in the
scene.
    // ...
    // Ball is in contact with Object.
    // ...

    // Make sure that the ball is actually headed towards the object. At
grazing angles there
    // could appear to be an impact when the ball is actually already hit
and moving away.

    if (impact > 0.0f)
    {
        ...
        // Play the sound associated with the Ammo hitting something.
        m_objects[i]->PlaySound(impact, m_player->Position());

        if (m_objects[i]->Target() && !m_objects[i]->Hit())
        {
            // The object is a target and isn't currently hit, so mark
            // it as hit and play the sound associated with the impact.
            m_objects[i]->Hit(true);
        }
    }
}
```



```

        m_objects[i]->HitTime(timeTotal);
        m_totalHits++;

        m_objects[i]->PlaySound(impact, m_player->Position());
    }
    ...
}
#pragma endregion

#pragma region Apply Gravity and world intersection
    // Apply gravity and check for collision against enclosing
    volume.
    ...
    if (position.z < limit)
    {
        // The ammo instance hit the a wall in the min Z
        direction.
        // Align the ammo instance to the wall, invert the Z
        component of the velocity and
        // play the impact sound.
        position.z = limit;
        m_ammo[i]->PlaySound(-velocity.z, m_player->Position());
        velocity.z = -velocity.z *
        GameConstants::Physics::GroundRestitution;
    }
    ...
#pragma endregion
}

```

Next steps

We have covered the UWP framework, graphics, controls, user interface, and audio of a Windows 10 game. The next part of this tutorial, [Extending the sample game](#), explains other options that can be used when developing a game.

Audio concepts

For Windows 10 games development, use XAudio2 version 2.9. This version is shipped with Windows 10. For more info, go to [XAudio2 Versions](#).

AudioX2 is a low-level API that provides signal processing and mixing foundation. For more info, see [XAudio2 Key Concepts](#).

XAudio2 voices

There are three types of XAudio2 voice objects: source, submix, and mastering voices. Voices are the objects XAudio2 use to process, to manipulate, and to play audio data.

- Source voices operate on audio data provided by the client.
- Source and submix voices send their output to one or more submix or mastering voices.
- Submix and mastering voices mix the audio from all voices feeding them, and operate on the result.
- Mastering voices receive data from source voices and submix voices, and sends that data to the audio hardware.

For more info, go to [XAudio2 voices](#).

Audio graph

Audio graph is a collection of [XAudio2 voices](#). Audio starts at one side of an audio graph in source voices, optionally passes through one or more submix voices, and ends at a mastering voice. An audio graph will contain a source voice for each sound currently playing, zero or more submix voices, and one mastering voice. The simplest audio graph, and the minimum needed to make a noise in XAudio2, is a single source voice outputting directly to a mastering voice. For more info, go to [Audio graphs](#).

Additional reading

- [How to: Initialize XAudio2](#)
- [How to: Load Audio Data Files in XAudio2](#)
- [How to: Play a Sound with XAudio2](#)

Key audio .h files

Audio.h

C++/WinRT

```
// Audio:
// This class uses XAudio2 to provide sound output. It creates two
// engines - one for music and the other for sound effects - each as
// a separate mastering voice.
// The SuspendAudio and ResumeAudio methods can be used to stop
// and start all audio playback.

class Audio
{
public:
    Audio();
```

```

void Initialize();
void CreateDeviceIndependentResources();
IXAudio2* MusicEngine();
IXAudio2* SoundEffectEngine();
void SuspendAudio();
void ResumeAudio();

private:
    ...
};

```

MediaReader.h

C++/WinRT

```

// MediaReader:
// This is a helper class for the SoundEffect class. It reads small audio
// files
// synchronously from the package installed folder and returns sound data as
// a
// vector of bytes.

class MediaReader
{
public:
    MediaReader();

    std::vector<byte> LoadMedia(_In_ winrt::hstring const& filename);
    WAVEFORMATEX* GetOutputWaveFormatEx();

private:
    winrt::Windows::Storage::StorageFolder m_installedLocation{ nullptr };
    winrt::hstring m_installedLocationPath;
    WAVEFORMATEX m_waveFormat;
};

```

SoundEffect.h

C++/WinRT

```

// SoundEffect:
// This class plays a sound using XAudio2. It uses a mastering voice
// provided
// from the Audio class. The sound data can be read from disk using the
// MediaReader
// class.

class SoundEffect
{

```

```
public:
    SoundEffect();

    void Initialize(
        _In_ IXAudio2* masteringEngine,
        _In_ WAVEFORMATEX* sourceFormat,
        _In_ std::vector<byte> const& soundData
    );

    void PlaySound(_In_ float volume);

private:
    ...
};
```

Extend the sample game

Article • 10/20/2022

ⓘ Note

This topic is part of the [Create a simple Universal Windows Platform \(UWP\) game with DirectX](#) tutorial series. The topic at that link sets the context for the series.

To download the version of this game that uses XAML for the overlay, see [DirectX and XAML game sample](#). Be sure to read the readme file there for details about building the sample.

At this point we've covered the key components of a basic Universal Windows Platform (UWP) DirectX 3D game. You can set up the framework for a game, including the view-provider and rendering pipeline, and implement a basic game loop. You can also create a basic user interface overlay, incorporate sounds, and implement controls. You're on your way to creating a game of your own, but if you need more help and info, check out these resources.

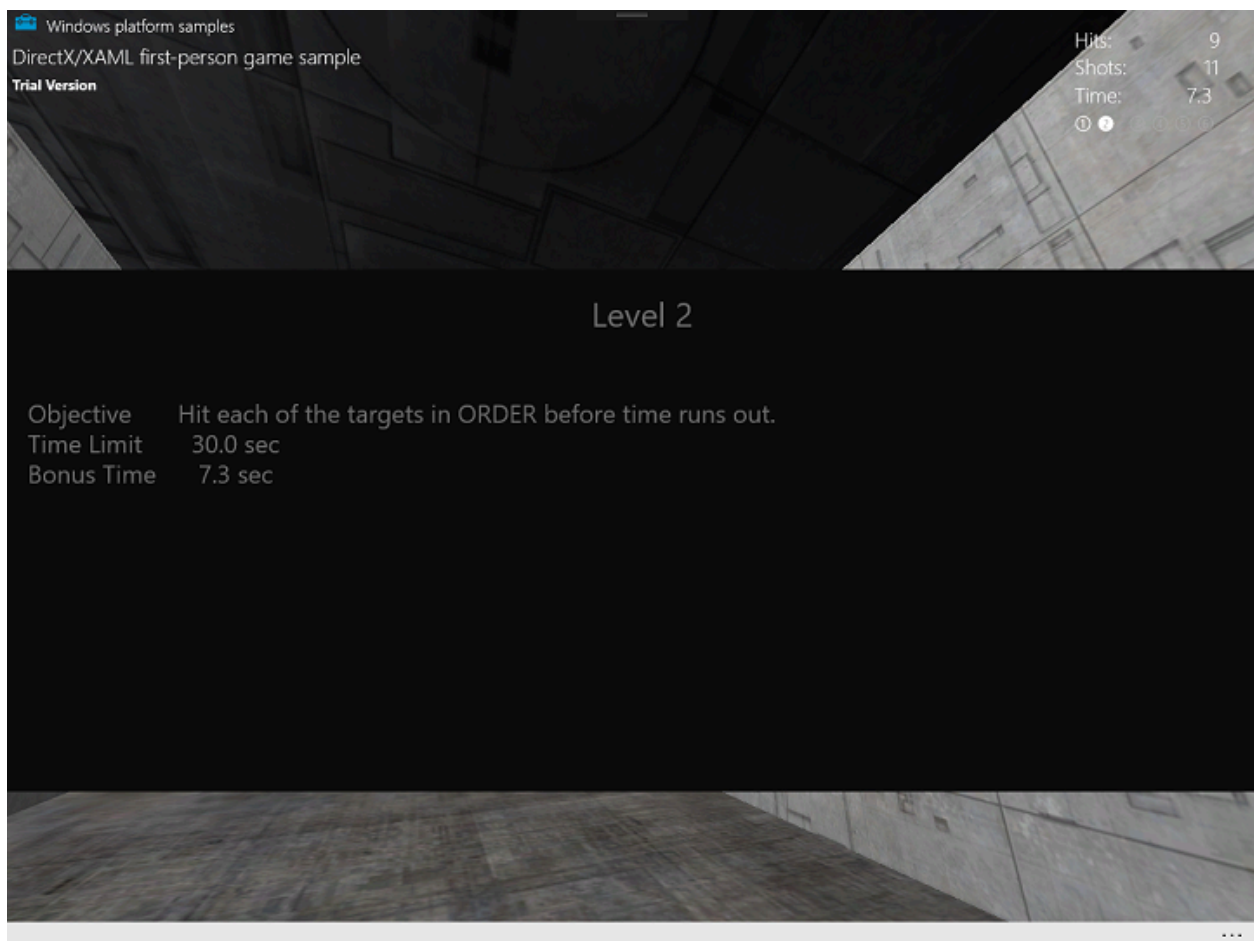
- [DirectX Graphics and Gaming](#)
- [Direct3D 11 Overview](#)
- [Direct3D 11 Reference](#)

Using XAML for the overlay

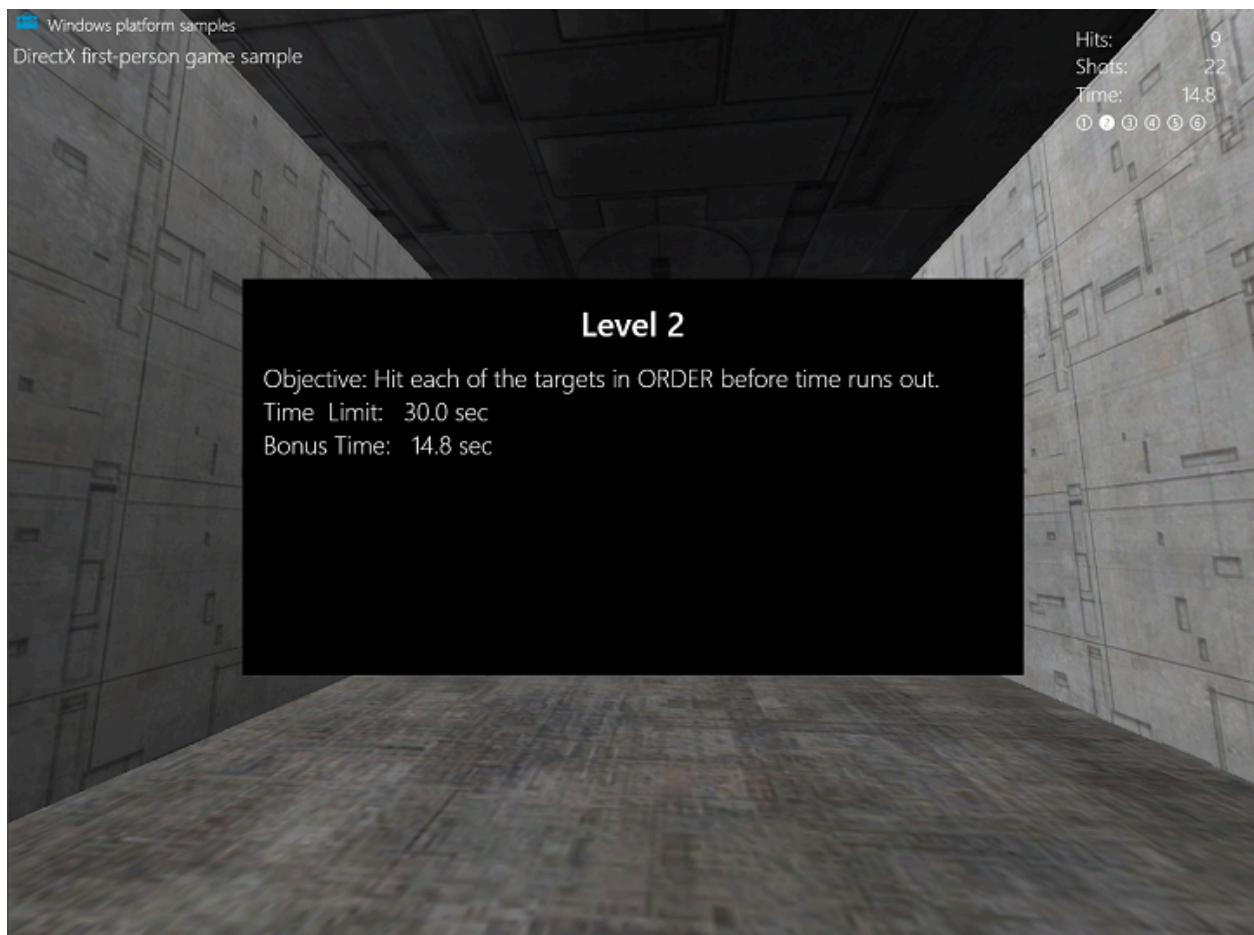
One alternative that we didn't discuss in depth is the use of XAML instead of [Direct2D](#) for the overlay. XAML has many benefits over Direct2D for drawing user interface elements. The most important benefit is that it makes incorporating the Windows 10 look and feel into your DirectX game more convenient. Many of the common elements, styles, and behaviors that define a UWP app are tightly integrated into the XAML model, making it far less work for a game developer to implement. If your own game design has a complicated user interface, consider using XAML instead of Direct2D.

With XAML, we can make a game interface that looks similar to the Direct2D one made earlier.

XAML



Direct2D



While they have similar end results, there are a number of differences between implementing Direct2D and XAML interfaces.

[Expand table](#)

Feature	XAML	Direct2D
Defining overlay	Defined in a XAML file, <code>*.xaml</code> . Once understanding XAML, creating and configuring more complicated overlays are made simpler when compared to Direct2D.	Defined as a collection of Direct2D primitives and DirectWrite strings manually placed and written to a Direct2D target buffer.
User interface elements	XAML user interface elements come from standardized elements that are part of the Windows Runtime XAML APIs, including Windows::UI::Xaml and Windows::UI::Xaml::Controls . The code that handles the behavior of the XAML user interface elements is defined in a codebehind file, <code>Main.xaml.cpp</code> .	Simple shapes can be drawn like rectangles and ellipses.
Window resizing	Naturally handles resize and view state change events, transforming the overlay accordingly	Need to manually specify how to redraw the overlay's components.

Another big difference involves the [swap chain](#). You don't have to attach the swap chain to a [Windows::UI::Core::CoreWindow](#) object. Instead, a DirectX app that incorporates XAML associates a swap chain when a new [SwapChainPanel](#) object is constructed.

The following snippet show how to declare XAML for the [SwapChainPanel](#) in the [DirectXPage.xaml](#) [file](#).

XML

```
<Page
  x:Class="Simple3DGameXaml.DirectXPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:Simple3DGameXaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  mc:Ignorable="d">
  <SwapChainPanel x:Name="DXSwapChainPanel">

    <!-- ... XAML user controls and elements -->

  </SwapChainPanel>
</Page>
```

The **SwapChainPanel** object is set as the **Content** property of the current window object created [at launch](#) by the app singleton.

C++

```
void App::OnLaunched(_In_ LaunchActivatedEventArgs^ /* args */)
{
    m_mainPage = ref new DirectXPage();

    Window::Current->Content = m_mainPage;
    // Bring the application to the foreground so that it's visible
    Window::Current->Activate();
}
```

To attach the configured swap chain to the **SwapChainPanel** instance defined by your XAML, you must obtain a pointer to the underlying native **ISwapChainPanelNative** interface implementation and call **ISwapChainPanelNative::SetSwapChain** on it, passing it your configured swap chain.

The following snippet from

[DX::DeviceResources::CreateWindowSizeDependentResources](#) details this for DirectX/XAML interop:

C++

```
ComPtr<IDXGIDevice3> dxgiDevice;
DX::ThrowIfFailed(
    m_d3dDevice.As(&dxgiDevice)
);

ComPtr<IDXGIAdapter> dxgiAdapter;
DX::ThrowIfFailed(
    dxgiDevice->GetAdapter(&dxgiAdapter)
);

ComPtr<IDXGIFactory2> dxgiFactory;
DX::ThrowIfFailed(
    dxgiAdapter->GetParent(IID_PPV_ARGS(&dxgiFactory))
);

// When using XAML interop, the swap chain must be created for
composition.
DX::ThrowIfFailed(
    dxgiFactory->CreateSwapChainForComposition(
        m_d3dDevice.Get(),
        &swapChainDesc,
        nullptr,
        &m_swapChain
    )
);
```



```

        // Associate swap chain with SwapChainPanel
        // UI changes will need to be dispatched back to the UI thread
        m_swapChainPanel->Dispatcher->RunAsync(CoreDispatcherPriority::High,
ref new DispatchedHandler([=]()
    {
        // Get backing native interface for SwapChainPanel
        ComPtr<ISwapChainPanelNative> panelNative;
        DX::ThrowIfFailed(
            reinterpret_cast<IUnknown*>(m_swapChainPanel)-
>QueryInterface(IID_PPV_ARGS(&panelNative))
        );
        DX::ThrowIfFailed(
            panelNative->SetSwapChain(m_swapChain.Get())
        );
    }, CallbackContext::Any));

        // Ensure that DXGI does not queue more than one frame at a time.
        // This both reduces latency and
        // ensures that the application will only render after each VSync,
        // minimizing power consumption.
        DX::ThrowIfFailed(
            dxgiDevice->SetMaximumFrameLatency(1)
        );
    }

```

For more info about this process, see [DirectX and XAML interop](#).

Sample

To download the version of this game that uses XAML for the overlay, see [DirectX and XAML game sample](#). Be sure to read the readme file there for details about building the sample.

Unlike the version of the sample game discussed in the rest of these topics, the XAML version defines its framework in the [App.xaml.cpp](#) and [DirectXPage.xaml.cpp](#) files, instead of [App.cpp](#) and [GameInfoOverlay.cpp](#), respectively.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Developing *Marble Maze*—a Universal Windows Platform (UWP) game built with C++ for DirectX

Article • 10/20/2022

This topic describes how to use DirectX and C++ to create a 3D Universal Windows Platform (UWP) game. The game, called *Marble Maze*, embraces multiple form factors such as tablets, traditional desktop PCs, and laptop PCs.

📌 Note

To download the *Marble Maze* source code, see the [sample on GitHub](#).

📌 Important

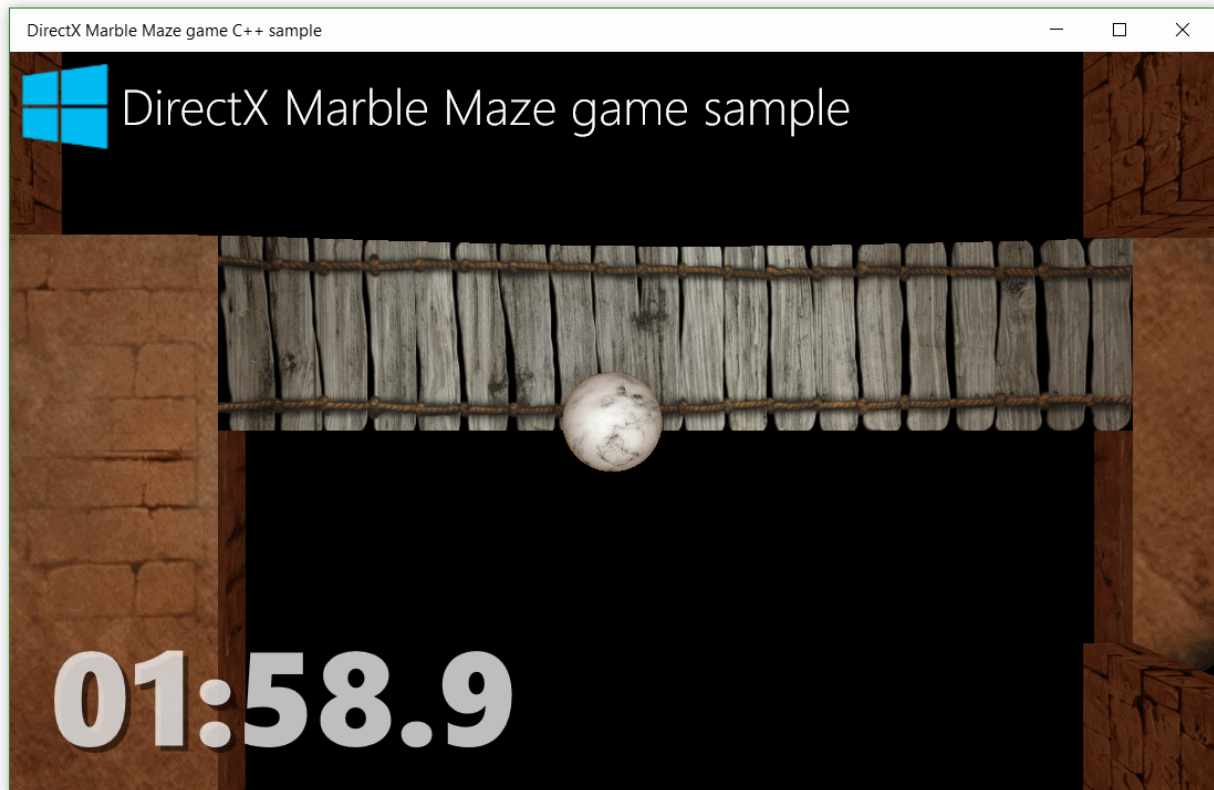
Marble Maze illustrates design patterns that we consider to be best practices for creating UWP games. You can adapt many of the implementation details to fit your own practices and the unique requirements of the game you are developing. Feel free to use different techniques or libraries when those better suit your needs. (However, always ensure that your code passes the [Windows App Certification Kit](#).) When we consider an implementation used here to be essential for successful game development, we emphasize it in this documentation.

Introducing *Marble Maze*

We chose *Marble Maze* because it is relatively basic, but still demonstrates the breadth of features that are found in most games. It shows how to use graphics, input handling, and audio. It also demonstrates game mechanics such as rules and goals.

Marble Maze resembles the table-top labyrinth game that is typically constructed from a box that contains holes and a steel or glass marble. The goal of *Marble Maze* is the same as the table-top version: tilt the maze to guide the marble from the start to the end of the maze in as little time as possible, without letting the marble fall into any of the holes. *Marble Maze* adds the concept of checkpoints. If the marble falls into a hole, the game is restarted at the last checkpoint location that the marble passed over.

Marble Maze offers multiple ways for a user to interact with the game board. If you have a touch-enabled or accelerometer-enabled device, you can use those devices to move the game board. You can also use a controller or a mouse to control game play.



Prerequisites

- Windows 10 Creators Update
- [Microsoft Visual Studio 2017](#) 
- C++ programming knowledge
- Familiarity with DirectX and DirectX terminology
- Basic knowledge of COM

Who should read this?

If you're interested in creating 3D games or other graphics-intensive applications for Windows 10, this is for you. We hope you use the principles and practices that this documentation outlines to create your own UWP game. A background or strong interest in C++ and DirectX programming will help you get the most out of this documentation. If you don't have experience with DirectX, you can still benefit if you have experience with similar 3D graphics programming environments.

The document [Walkthrough: create a simple UWP game with DirectX](#) describes another sample that implements a basic 3D shooting game by using DirectX and C++.

What this documentation covers

This documentation teaches how to:

- Use the Windows Runtime API and DirectX to create a UWP game.
- Use [Direct3D](#) and [Direct2D](#) to work with visual content such as models, textures, vertex and pixel shaders, and 2D overlays.
- Integrate input mechanisms such as touch, accelerometer, and game controller.
- Use [XAudio2](#) to incorporate music and sound effects.

What this documentation does not cover

This documentation does not cover the following aspects of game development. These aspects are followed by additional resources that cover them.

- 3D game design principles.
- C++ or DirectX programming basics.
- How to design resources such as textures, models, or audio.
- How to troubleshoot behavior or performance issues in your game.
- How to prepare your game for use in other parts of the world.
- How to certify and publish your game to the Microsoft Store.

Marble Maze also uses the [DirectXMath](#) library to work with 3D geometry and perform physics calculations, such as collisions. DirectXMath is not covered in-depth in this section. For details about how *Marble Maze* uses DirectXMath, refer to the source code.

Although *Marble Maze* provides many reusable components, it is not a complete game development framework. When we consider a *Marble Maze* component to be reusable in your game, we emphasize it in the documentation.

Next steps

We recommend that you start with [Marble Maze sample fundamentals](#) to learn about the *Marble Maze* structure and some of the coding and style guidelines that the *Marble Maze* source code follows. The following table outlines the documents in this section so that you can more easily refer to them.

In this section

Title	Description
Marble Maze sample fundamentals	Provides an overview of the game structure and some of the code and style guidelines that the source code follows.
Marble Maze application structure	Describes how the <i>Marble Maze</i> application code is structured and how the structure of a DirectX UWP app differs from that of a traditional desktop application.
Adding visual content to the Marble Maze sample	Describes some of the key practices to keep in mind when you work with Direct3D and Direct2D. Also describes how <i>Marble Maze</i> applies these practices for visual content.
Adding input and interactivity to the Marble Maze sample	Describes how <i>Marble Maze</i> works with accelerometer, touch, and game controller inputs to enable users to navigate menus and interact with the game board. Also describes some of the best practices to keep in mind when you work with input.
Adding audio to the Marble Maze sample	Describes how <i>Marble Maze</i> works with audio to add music and sound effects to the game experience.

Marble Maze sample fundamentals

Article • 10/20/2022

This topic describes the fundamental characteristics of the Marble Maze project—for example, how it uses Visual C++ in the Windows Runtime environment, how it is created and structured, and how it is built. The topic also describes several of the conventions that are used in the code.

⚠ Note

The sample code that corresponds to this document is found in the [DirectX Marble Maze game sample](#) [↗].

Here are some of the key points that this document discusses for when you plan and develop your Universal Windows Platform (UWP) game.

- Use the **DirectX 11 App (Universal Windows - C++/CX)** template in Visual Studio to create your DirectX UWP game.
- The Windows Runtime provides classes and interfaces so that you can develop UWP apps in a more modern, object-oriented manner.
- Use object references with the hat (^) symbol to manage the lifetime of Windows Runtime variables, [Microsoft::WRL::ComPtr](#) to manage the lifetime of COM objects, and [std::shared_ptr](#) or [std::unique_ptr](#) to manage the lifetime of all other heap-allocated C++ objects.
- In most cases, use exception handling, instead of result codes, to deal with unexpected errors.
- Use [SAL annotations](#) together with code analysis tools to help discover errors in your app.

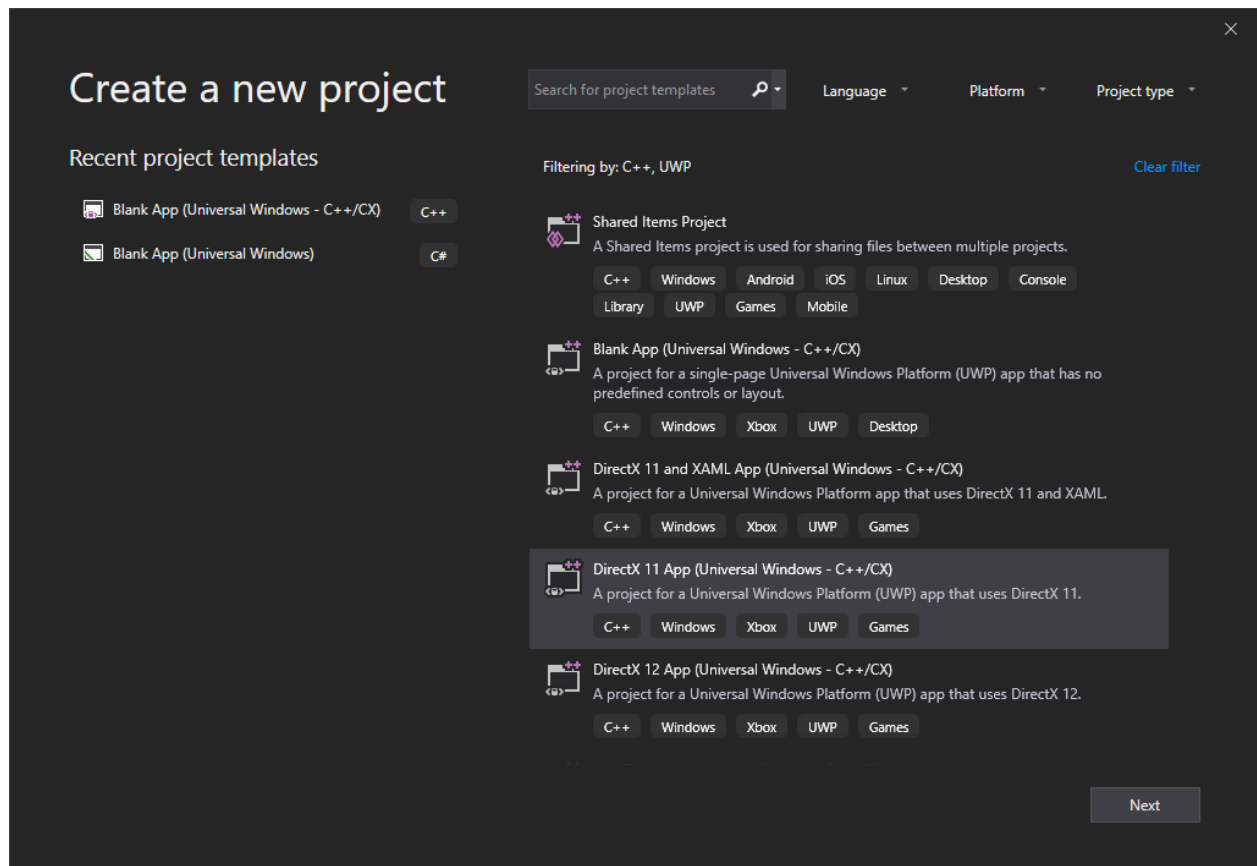
Creating the Visual Studio project

If you've downloaded and extracted the sample, you can open the **MarbleMaze_VS2017.sln** file (in the C++ folder) in Visual Studio, and you'll have the code in front of you.

When we created the Visual Studio project for Marble Maze, we started with an existing project. However, if you do not already have an existing project that provides the basic functionality that your DirectX UWP game requires, we recommend that you create a project based on the Visual Studio **DirectX 11 App (Universal Windows - C++/CX)**

template because it provides a basic working 3D application. To do this, follow these steps:

1. In Visual Studio 2019, select **File > New > Project...**
2. In the **Create a new project** window, select **DirectX 11 App (Universal Windows - C++/CX)**. If you don't see this option, you may not have the required components installed—see [Modify Visual Studio 2019 by adding or removing workloads and components](#) for information about how to install additional components.



3. Select **Next**, and then enter a **Project name**, a **Location** for the files to be stored, and a **Solution name**, and then select **Create**.

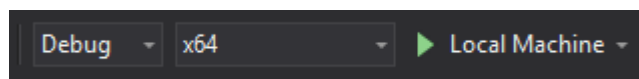
One important project setting in the **DirectX 11 App (Universal Windows - C++/CX)** template is the **/ZW** option, which enables the program to use the Windows Runtime language extensions. This option is enabled by default when you use the Visual Studio template. See [Compiler and Linker options \(C++/CX\)](#) for more info about how to set compiler options in Visual Studio.

Caution The **/ZW** option is not compatible with options such as **/clr**. In the case of **/clr**, this means that you cannot target both the .NET Framework and the Windows Runtime from the same Visual C++ project.

Every UWP app that you acquire from the Microsoft Store comes in the form of an app package. An app package contains a package manifest, which contains information about your app. For example, you can specify the capabilities (that is, the required access to protected system resources or user data) of your app. If you determine that your app requires certain capabilities, use the package manifest to declare the required capabilities. The manifest also lets you specify project properties such as supported device rotations, tile images, and the splash screen. You can edit the manifest by opening **Package.appxmanifest** in your project. For more info about app packages, see [Packaging apps](#).

Building, deploying, and running the game

In the dropdown menus at the top of Visual Studio, to the left of the green play button, select your deployment configuration. We recommend setting it as **Debug** targeting your device's architecture (**x86** for 32-bit, **x64** for 64-bit) and to your **Local Machine**. You can also test on a **Remote Machine**, or to a **Device** that's connected via USB. Then click the green play button to build and deploy to your device.



Controlling the game

You can use touch, the accelerometer, a game controller, or the mouse to control Marble Maze.

- Use the directional pad on the controller to change the active menu item.
- Use touch, the A or Start button on the controller, or the mouse to pick a menu item.
- Use touch, the accelerometer, the left thumbstick, or the mouse to tilt the maze.
- Use touch, the A or Start button on the controller, or the mouse to close menus such as the high score table.
- Use the Start button on the controller or the P key on the keyboard to pause or resume the game.
- Use the Back button on the controller or the Home key on the keyboard to restart the game.
- When the high-score table is visible, use the Back button on the controller or the Home key on the keyboard to clear all scores.

Code conventions

The Windows Runtime is a programming interface that you can use to create UWP apps that run only in a special application environment. Such apps use authorized functions, data types, and devices, and are distributed from the Microsoft Store. At the lowest level, the Windows Runtime consists of an Application Binary Interface (ABI). The ABI is a low-level binary contract that makes Windows Runtime APIs accessible to multiple programming languages such as JavaScript, the .NET languages, and Visual C++.

In order to call Windows Runtime APIs from JavaScript and .NET, those languages require projections that are specific to each language environment. When you call a Windows Runtime API from JavaScript or .NET, you are invoking the projection, which in turn calls the underlying ABI function. Although you can call the ABI functions directly in C++, Microsoft provides projections for C++ as well, because they make it much simpler to consume the Windows Runtime APIs, while still maintaining high performance. Microsoft also provides language extensions to Visual C++ that specifically support the Windows Runtime projections. Many of these language extensions resemble the syntax for the C++/CLI language. However, instead of targeting the common language runtime (CLR), native apps use this syntax to target the Windows Runtime. The object reference, or hat (^), modifier is an important part of this new syntax because it enables the automatic deletion of runtime objects by means of reference counting. Instead of calling methods such as [AddRef](#) and [Release](#) to manage the lifetime of a Windows Runtime object, the runtime deletes the object when no other component references it, for example, when it leaves scope or you set all references to **nullptr**. Another important part of using Visual C++ to create UWP apps is the **ref new** keyword. Use **ref new** instead of **new** to create reference-counted Windows Runtime objects. For more info, see [Type System \(C++/CX\)](#).

Important

You only have to use ^ and **ref new** when you create Windows Runtime objects or create Windows Runtime components. You can use the standard C++ syntax when you write core application code that does not use the Windows Runtime.

Marble Maze uses ^ together with **Microsoft::WRL::ComPtr** to manage heap-allocated objects and minimize memory leaks. We recommend that you use ^ to manage the lifetime of Windows Runtime variables, **ComPtr** to manage the lifetime of COM variables (such as when you use DirectX), and **std::shared_ptr** or **std::unique_ptr** to manage the lifetime of all other heap-allocated C++ objects.

For more info about the language extensions that are available to a C++ UWP app, see [Visual C++ Language Reference \(C++/CX\)](#).

Error handling

Marble Maze uses exception handling as the primary way to deal with unexpected errors. Although game code traditionally uses logging or error codes, such as **HRESULT** values, to indicate errors, exception handling has two main advantages. First, it can make the code easier to read and maintain. From a code perspective, exception handling is a more efficient way to propagate an error to a routine that can handle that error. The use of error codes typically requires each function to explicitly propagate errors. A second advantage is that you can configure the Visual Studio debugger to break when an exception occurs so that you can stop immediately at the location and context of the error. The Windows Runtime also uses exception handling extensively. Therefore, by using exception handling in your code, you can combine all error handling into one model.

We recommend that you use the following conventions in your error handling model:

- Use exceptions to communicate unexpected errors.
- Do not use exceptions to control the flow of code.
- Catch only the exceptions that you can safely handle and recover from. Otherwise, do not catch the exception and allow the app to terminate.
- When you call a DirectX routine that returns **HRESULT**, use the **DX::ThrowIfFailed** function. This function is defined in [DirectXHelper.h](#). **ThrowIfFailed** throws an exception if the provided **HRESULT** is an error code. For example, **E_POINTER** causes **ThrowIfFailed** to throw [Platform::NullReferenceException](#).

When you use **ThrowIfFailed**, put the DirectX call on a separate line to help improve code readability, as shown in the following example.

C++

```
// Identify the physical adapter (GPU or card) this device is running on.
ComPtr<IDXGIAdapter> dxgiAdapter;
DX::ThrowIfFailed(
    dxgiDevice->GetAdapter(&dxgiAdapter)
);
```

- Although we recommend that you avoid the use of **HRESULT** for unexpected errors, it is more important to avoid the use of exception handling to control the flow of code. Therefore, it is preferred to use an **HRESULT** return value when necessary to control the flow of code.

SAL annotations

Use SAL annotations together with code analysis tools to help discover errors in your app.

By using Microsoft source-code annotation language (SAL), you can annotate, or describe, how a function uses its parameters. SAL annotations also describe return values. SAL annotations work with the C/C++ Code Analysis tool to discover possible defects in C and C++ source code. Common coding errors reported by the tool include buffer overruns, uninitialized memory, null pointer dereferences, and memory and resource leaks.

Consider the **BasicLoader::LoadMesh** method, which is declared in [BasicLoader.h](#). This method uses `_In_` to specify that *filename* is an input parameter (and therefore will only be read from), `_Out_` to specify that *vertexBuffer* and *indexBuffer* are output parameters (and therefore will only be written to), and `_Out_opt_` to specify that *vertexCount* and *indexCount* are optional output parameters (and might be written to). Because *vertexCount* and *indexCount* are optional output parameters, they are allowed to be **nullptr**. The C/C++ Code Analysis tool examines calls to this method to ensure that the parameters it passes meet these criteria.

C++

```
void LoadMesh(  
    _In_ Platform::String^ filename,  
    _Out_ ID3D11Buffer** vertexBuffer,  
    _Out_ ID3D11Buffer** indexBuffer,  
    _Out_opt_ uint32* vertexCount,  
    _Out_opt_ uint32* indexCount  
);
```

To perform code analysis on your app, on the menu bar, choose **Build > Run Code Analysis on Solution**. For more info about code analysis, see [Analyzing C/C++ Code Quality by Using Code Analysis](#).

The complete list of available annotations is defined in `sal.h`. For more info, see [SAL Annotations](#).

Next steps

Read [Marble Maze application structure](#) for information about how the Marble Maze application code is structured and how the structure of a DirectX UWP app differs from that of a traditional desktop application.

Related topics

- [Marble Maze application structure](#)
- [Developing Marble Maze, a UWP game in C++ and DirectX](#)

Marble Maze application structure

Article • 10/20/2022

The structure of a DirectX Universal Windows Platform (UWP) app differs from that of a traditional desktop application. Instead of working with handle types such as [HWND](#) and functions such as [CreateWindow](#), the Windows Runtime provides interfaces such as [Windows::UI::Core::ICoreWindow](#) so that you can develop UWP apps in a more modern, object-oriented manner. This section of the documentation shows how the Marble Maze app code is structured.

ⓘ Note

The sample code that corresponds to this document is found in the [DirectX Marble Maze game sample](#).

Here are some of the key points that this document discusses for when you structure your game code:

- In the initialization phase, set up the runtime and library components that your game uses, and load game-specific resources.
- UWP apps must start processing events within 5 seconds of launch. Therefore, load only essential resources when you load your app. Games should load large resources in the background and display a progress screen.
- In the game loop, respond to Windows events, read user input, update scene objects, and render the scene.
- Use event handlers to respond to window events. (These replace the window messages from desktop Windows applications.)
- Use a state machine to control the flow and order of the game logic.

File organization

Some of the components in Marble Maze can be reused with any game with little or no modification. For your own game, you can adapt the organization and ideas that these files provide. The following table briefly describes the important source code files.

Files	Description
App.h, App.cpp	Defines the App and DirectXApplicationSource classes, which encapsulate the view (window, thread, and events) of the app.

Files	Description
Audio.h, Audio.cpp	Defines the Audio class, which manages audio resources.
BasicLoader.h, BasicLoader.cpp	Defines the BasicLoader class, which provides utility methods that help you load textures, meshes, and shaders.
BasicMath.h	Defines structures and functions that help you work with vector and matrix data and computations. Many of these functions are compatible with HLSL shader types.
BasicReaderWriter.h, BasicReaderWriter.cpp	Defines the BasicReaderWriter class, which uses the Windows Runtime to read and write file data in a UWP app.
BasicShapes.h, BasicShapes.cpp	Defines the BasicShapes class, which provides utility methods for creating basic shapes such as cubes and spheres. (These files are not used by the Marble Maze implementation).
Camera.h, Camera.cpp	Defines the Camera class, which provides the position and orientation of a camera.
Collision.h, Collision.cpp	Manages collision info between the marble and other objects, such as the maze.
DDSTextureLoader.h, DDSTextureLoader.cpp	Defines the CreateDDSTextureFromMemory function, which loads textures that are in .dds format from a memory buffer.
DirectXHelper.h	Defines DirectX helper functions that are useful to many DirectX UWP apps.
LoadScreen.h, LoadScreen.cpp	Defines the LoadScreen class, which displays a loading screen during app initialization.
MarbleMazeMain.h, MarbleMazeMain.cpp	Defines the MarbleMazeMain class, which manages game-specific resources and defines much of the game logic.
MediaStreamer.h, MediaStreamer.cpp	Defines the MediaStreamer class, which uses Media Foundation to help the game manage audio resources.
PersistentState.h, PersistentState.cpp	Defines the PersistentState class, which reads and writes primitive data types from and to a backing store.
Physics.h, Physics.cpp	Defines the Physics class, which implements the physics simulation between the marble and the maze.
Primitives.h	Defines geometric types that are used by the game.
SampleOverlay.h, SampleOverlay.cpp	Defines the SampleOverlay class, which provides common 2D and user-interface data and operations.
SDKMesh.h, SDKMesh.cpp	Defines the SDKMesh class, which loads and renders meshes that are in SDK Mesh (.sdkmesh) format.

Files	Description
StepTimer.h	Defines the StepTimer class, which provides an easy way to get total and elapsed times.
UserInterface.h, UserInterface.cpp	Defines functionality that's related to the user interface, such as the menu system and the high score table.

Design-time versus run-time resource formats

When you can, use run-time formats instead of design-time formats to more efficiently load game resources.

A *design-time* format is the format you use when you design your resource. Typically, 3D designers work with design-time formats. Some design-time formats are also text-based so that you can modify them in any text-based editor. Design-time formats can be verbose and contain more information than your game requires. A *run-time* format is the binary format that is read by your game. Run-time formats are typically more compact and more efficient to load than the corresponding design-time formats. This is why the majority of games use run-time assets at run time.

Although your game can directly read a design-time format, there are several benefits to using a separate run-time format. Because run-time formats are often more compact, they require less disk space and require less time to transfer over a network. Also, run-time formats are often represented as memory-mapped data structures. Therefore, they can be loaded into memory much faster than, for example, an XML-based text file. Finally, because separate run-time formats are typically binary-encoded, they are more difficult for the end-user to modify.

HLSL shaders are one example of resources that use different design-time and run-time formats. Marble Maze uses .hlsl as the design-time format, and .cso as the run-time format. A .hlsl file holds source code for the shader; a .cso file holds the corresponding shader byte code. When you convert .hlsl files offline and provide .cso files with your game, you avoid the need to convert HLSL source files to byte code when your game loads.

For instructional reasons, the Marble Maze project includes both the design-time format and the run-time format for many resources, but you only have to maintain the design-time formats in the source project for your own game because you can convert them to run-time formats when you need them. This documentation shows how to convert the design-time formats to the run-time formats.

Application life cycle

Marble Maze follows the life cycle of a typical UWP app. For more info about the life cycle of a UWP app, see [App lifecycle](#).

When a UWP game initializes, it typically initializes runtime components such as Direct3D, Direct2D, and any input, audio, or physics libraries that it uses. It also loads game-specific resources that are required before the game begins. This initialization occurs one time during a game session.

After initialization, games typically run the *game loop*. In this loop, games typically perform four actions: process Windows events, collect input, update scene objects, and render the scene. When the game updates the scene, it can apply the current input state to the scene objects and simulate physical events, such as object collisions. The game can also perform other activities such as playing sound effects or sending data over the network. When the game renders the scene, it captures the current state of the scene and draws it to the display device. The following sections describe these activities in greater detail.

Adding to the template

The **DirectX 11 App (Universal Windows)** template creates a core window to which you can render with Direct3D. The template also includes the **DeviceResources** class that creates all of the Direct3D device resources needed for rendering 3D content in a UWP app.

The **App** class creates the **MarbleMazeMain** class object, starts the loading of resources, loops to update the timer, and calls the **MarbleMazeMain::Render** method each frame. The **App::OnWindowSizeChanged**, **App::OnDpiChanged**, and **App::OnOrientationChanged** methods each call the **MarbleMazeMain::CreateWindowSizeDependentResources** method, and the **App::Run** method calls the **MarbleMazeMain::Update** and **MarbleMazeMain::Render** methods.

The following example shows where the **App::SetWindow** method creates the **MarbleMazeMain** class object. The **DeviceResources** class is passed to the method so it can use the Direct3D objects for rendering.

C++

```
m_main = std::unique_ptr<MarbleMazeMain>(new  
MarbleMazeMain(m_deviceResources));
```


The **App** class also starts loading the deferred resources for the game. See the next section for more detail.

Additionally, the **App** class sets up the event handlers for the [CoreWindow](#) events. When the handlers for these events are called, they pass the input to the **MarbleMazeMain** class.

Loading game assets in the background

To ensure that your game can respond to window events within 5 seconds after it is launched, we recommend that you load your game assets asynchronously, or in the background. As assets load in the background, your game can respond to window events.

ⓘ Note

You can also display the main menu when it is ready, and allow the remaining assets to continue loading in the background. If the user selects an option from the menu before all resources are loaded, you can indicate that scene resources are continuing to load by displaying a progress bar, for example.

Even if your game contains relatively few game assets, it is good practice to load them asynchronously for two reasons. One reason is that it is difficult to guarantee that all of your resources will load quickly on all devices and all configurations. Also, by incorporating asynchronous loading early, your code is ready to scale as you add functionality.

Asynchronous asset loading begins with the **App::Load** method. This method uses the [task](#) class to load game assets in the background.

C++

```
task<void>([=]()
{
    m_main->LoadDeferredResources(true, false);
});
```

The **MarbleMazeMain** class defines the *m_deferredResourcesReady* flag to indicate that asynchronous loading is complete. The **MarbleMazeMain::LoadDeferredResources** method loads the game resources and then sets this flag. The update

(**MarbleMazeMain::Update**) and render (**MarbleMazeMain::Render**) phases of the app check this flag. When this flag is set, the game continues as normal. If the flag is not yet set, the game shows the loading screen.

For more information about asynchronous programming for UWP apps, see [Asynchronous programming in C++](#).

Tip

If you're writing game code that is part of a Windows Runtime C++ Library (in other words, a DLL), consider whether to read **Creating Asynchronous Operations in C++ for UWP apps** to learn how to create asynchronous operations that can be consumed by apps and other libraries.

The game loop

The **App::Run** method runs the main game loop (**MarbleMazeMain::Update**). This method is called every frame.

To help separate view and window code from game-specific code, we implemented the **App::Run** method to forward update and render calls to the **MarbleMazeMain** object.

The following example shows the **App::Run** method, which includes the main game loop. The game loop updates the total time and frame time variables, and then updates and renders the scene. This also makes sure that content is only rendered when the window is visible.

C++

```
void App::Run()
{
    while (!m_windowClosed)
    {
        if (m_windowVisible)
        {
            CoreWindow::GetForCurrentThread()->Dispatcher->
                ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

            m_main->Update();

            if (m_main->Render())
            {
                m_deviceResources->Present();
            }
        }
    }
}
```

```

    }
}
else
{
    CoreWindow::GetForCurrentThread()->Dispatcher->
ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
}
}

// The app is exiting so do the same thing as if the app were being
suspended.
m_main->OnSuspending();

#ifdef _DEBUG
    // Dump debug info when exiting.
    DumpD3DDebug();
#endif // _DEBUG
}

```

The state machine

Games typically contain a *state machine* (also known as a *finite state machine*, or FSM) to control the flow and order of the game logic. A state machine contains a given number of states and the ability to transition among them. A state machine typically starts from an *initial* state, transitions to one or more *intermediate* states, and possibly ends at a *terminal* state.

A game loop often uses a state machine so that it can perform the logic that is specific to the current game state. Marble Maze defines the **GameState** enumeration, which defines each possible state of the game.

C++

```

enum class GameState
{
    Initial,
    MainMenu,
    HighScoreDisplay,
    PreGameCountdown,
    InGameActive,
    InGamePaused,
    PostGameResults,
};

```

The **MainMenu** state, for example, defines that the main menu appears, and that the game is not active. Conversely, the **InGameActive** state defines that the game is active,

and that the menu does not appear. The **MarbleMazeMain** class defines the **m_gameState** member variable to hold the active game state.

The **MarbleMazeMain::Update** and **MarbleMazeMain::Render** methods use switch statements to perform logic for the current state. The following example shows what a switch statement might look like for the **MarbleMazeMain::Update** method (details are removed to illustrate the structure).

C++

```
switch (m_gameState)
{
case GameState::MainMenu:
    // Do something with the main menu.
    break;

case GameState::HighScoreDisplay:
    // Do something with the high-score table.
    break;

case GameState::PostGameResults:
    // Do something with the game results.
    break;

case GameState::InGamePaused:
    // Handle the paused state.
    break;
}
```

When game logic or rendering depends on a specific game state, we emphasize it in this documentation.

Handling app and window events

The Windows Runtime provides an object-oriented event-handling system so that you can more easily manage Windows messages. To consume an event in an application, you must provide an event handler, or event-handling method, that responds to the event. You must also register the event handler with the event source. This process is often referred to as event wiring.

Supporting suspend, resume, and restart

Marble Maze is suspended when the user switches away from it or when Windows enters a low power state. The game is resumed when the user moves it to the foreground or when Windows comes out of a low power state. Generally, you don't

close apps. Windows can terminate the app when it's in the suspended state and Windows requires the resources, such as memory, that the app is using. Windows notifies an app when it is about to be suspended or resumed, but it doesn't notify the app when it's about to be terminated. Therefore, your app must be able to save—at the point when Windows notifies your app that it is about to be suspended—any data that would be required to restore the current user state when the app is restarted. If your app has significant user state that is expensive to save, you may also need to save state regularly, even before your app receives the suspend notification. Marble Maze responds to suspend and resume notifications for two reasons:

1. When the app is suspended, the game saves the current game state and pauses audio playback. When the app is resumed, the game resumes audio playback.
2. When the app is closed and later restarted, the game resumes from its previous state.

Marble Maze performs the following tasks to support suspend and resume:

- It saves its state to persistent storage at key points in the game, such as when the user reaches a checkpoint.
- It responds to suspend notifications by saving its state to persistent storage.
- It responds to resume notifications by loading its state from persistent storage. It also loads the previous state during startup.

To support suspend and resume, Marble Maze defines the **PersistentState** class. (See **PersistentState.h** and **PersistentState.cpp**). This class uses the [Windows::Foundation::Collections::IPropertySet](#) interface to read and write properties. The **PersistentState** class provides methods that read and write primitive data types (such as **bool**, **int**, **float**, **XMFLOAT3**, and [Platform::String](#)), from and to a backing store.

C++

```
ref class PersistentState
{
internal:
    void Initialize(
        _In_ Windows::Foundation::Collections::IPropertySet^ settingsValues,
        _In_ Platform::String^ key
    );

    void SaveBool(Platform::String^ key, bool value);
    void SaveInt32(Platform::String^ key, int value);
    void SaveSingle(Platform::String^ key, float value);
    void SaveXMFLOAT3(Platform::String^ key, DirectX::XMFLOAT3 value);
    void SaveString(Platform::String^ key, Platform::String^ string);

    bool LoadBool(Platform::String^ key, bool defaultValue);
    int LoadInt32(Platform::String^ key, int defaultValue);
```